



std::simd overloads for bit permutations

Document number:

[P3772R0](#)

Date:

2025-06-28

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3772/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/bit-permutations-simd.cow



ABSTRACT: [\[P3104R3\]](#) has been design-approved by LEWG for C++29. This proposal adds std::simd overloads to add consistency with [\[P2933R4\]](#).

§ Contents

- 1 Introduction
- 2 Design
- 3 Implementation experience
- 4 Wording
- 5 References

§ 1. Introduction

[\[P3104R3\]](#) has been design-approved by LEWG for C++29. During the discussion of the paper, it was *not* mentioned that [\[P2933R4\]](#) adds std::simd overloads for almost all functions

in the `<bit>` header.



For the purpose of consistency, this proposal adds these overloads.



NOTE: This proposal has an extremely minimal motivation and design discussion. That is because I see it as an obvious extensions of the existing design, and possibly a bug fix.

Should the need arise, these topics could be explored more deeply.

§ 2. Design

Some of the design in this proposal has direct equivalents:

- `std::simd::bit_repeat` is analogous to `std::simd::rotr`.
- `std::simd::bit_reverse` is analogous to `std::simd::byteswap`.

The signatures of the the new and respective existing functions are identical. `std::bit_compress` and `std::bit_expand` are binary operations between two unsigned integers of the same type, so naturally, `std::simd::bit_compress` and `std::simd::bit_expand` are binary operations between two `std::simd::vecs` of the same type.

§ 3. Implementation experience

None.

§ 4. Wording

In [\[version.syn\]](#), bump the feature-test macro:



```
#define __cpp_lib_simd 202506L 20XXXXL // also in <simd>
```

In [\[simd.syn\]](#), change the synopsis as follows:



```
namespace std::simd {  
    [...]  
  
    // [simd.bit], Bit manipulation  
    template<simd-type V> constexpr V byteswap(const V& v) noexcept;  
    template<simd-type V> constexpr V bit_reverse(const V& v) noexcept;  
    template<simd-type V> constexpr V bit_ceil(const V& v) noexcept;  
    template<simd-type V> constexpr V bit_floor(const V& v) noexcept;  
  
    template<simd-type V>  
        constexpr typename V::mask_type has_single_bit(const V& v) noexcept;
```



```
template<simd-type V0, simd-type V1>
    constexpr V0 rotl(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V rotl(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 rotr(const V0& v, const V1& s) noexcept;
template<simd-type V>
    constexpr V rotr(const V& v, int s) noexcept;

template<simd-type V0, simd-type V1>
    constexpr V0 bit_repeat(const V0& v, const V1& l) noexcept;
template<simd-type V>
    constexpr V bit_repeat(const V& v, int l) noexcept;

template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        bit_width(const V& v) noexcept;
template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        countl_zero(const V& v) noexcept;
template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        countl_one(const V& v) noexcept;
template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        countr_zero(const V& v) noexcept;
template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        countr_one(const V& v) noexcept;
template<simd-type V>
    constexpr rebind_t<make_signed_t<typename V::value_type>, V>
        popcount(const V& v) noexcept;

template<simd-type V>
    constexpr V bit_compress(const V& v, const V& m) noexcept;
template<simd-type V>
    constexpr V bit_expand(const V& v, const V& m) noexcept;

[...]
```

```
}

namespace std {
    [...]

    // [simd.bit], Bit manipulation
    using simd::byteswap;
    using simd::bit_ceil;
    using simd::bit_floor;
    using simd::bit_reverse;
    using simd::has_single_bit;
    using simd::rotl;
    using simd::rotr;
```



```

using simd::bit_repeat;
using simd::bit_width;
using simd::countl_zero;
using simd::countl_one;
using simd::countr_zero;
using simd::countr_one;
using simd::popcount;
using simd::bit_compress;
using simd::bit_expand;

[...]
}

```

Change `[simd.bit]` as follows:



basic_vec bit library

[simd.bit]

```
template<simd-type V> constexpr V byteswap(const V& v) noexcept;
```

1 *Constraints:* The type `V::value_type` models integral.

2 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `std::byteswap(v[i])` for all i in the range `[0, V::size())`.

```
template<simd-type V> constexpr V bit_reverse(const V& v) noexcept;
```

¶ *Constraints:* The type `V::value_type` is an unsigned integer type ([basic.fundamental]).

¶ *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `std::bit_reverse(v[i])` for all i in the range `[0, V::size())`.

```
template<simd-type V> constexpr V bit_ceil(const V& v) noexcept;
```

3 *Constraints:* The type `V::value_type` is an unsigned integer type ([basic.fundamental]).

4 *Preconditions:* For every i in the range `[0, V::size())`, the smallest power of 2 greater than or equal to `v[i]` is representable as a value of type `V::value_type`.

5 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `std::bit_ceil(v[i])` for all i in the range `[0, V::size())`.

6 *Remarks:* A function call expression that violates the precondition in the *Preconditions*: element is not a core constant expression ([expr.const]).

```
template<simd-type V> constexpr V bit_floor(const V& v) noexcept;
```

7 *Constraints:* The type `V::value_type` is an unsigned integer type ([basic.fundamental]).

8 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `std::bit_floor(v[i])` for all i in the range `[0, V::size())`.

```
template<simd-type V>
```

```
constexpr typename V::mask_type has_single_bit(const V& v) noexcept;
```



9 *Constraints:* The type `V::value_type` is an unsigned integer type ([[basic.fundamental](#)]).

10 *Returns:* A `basic_mask` object where the i^{th} element is initialized to the result of `std::has_single_bit(v[i])` for all i in the range `[0, V::size())`.

```
template<simd-type V0, simd-type V1>
```

```
constexpr V0 rotl(const V0& v0, const V1& v1) noexcept;
```

```
template<simd-type V0, simd-type V1>
```

```
constexpr V0 rotr(const V0& v0, const V1& v1) noexcept;
```

```
template<simd-type V0, simd-type V1>
```

```
constexpr V0 bit_repeat(const V0& v0, const V1& v1) noexcept;
```

11 *Constraints:*

- The type `V0::value_type` is an unsigned integer type ([[basic.fundamental](#)]),
- the type `V1::value_type` models integral,
- `V0::size() == V1::size()` is `true`, and
- `sizeof(typename V0::value_type) == sizeof(typename V1::value_type)` is `true`.

¶ *Preconditions:* For `bit_repeat`, `v1[i] > 0` is true for all i in the range `[0, V0::size())`.

12 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v0[i], static_cast<int>(v1[i]))` for all i in the range `[0, V0::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-type V> constexpr V rotl(const V& v, int s) noexcept;
```

```
template<simd-type V> constexpr V rotr(const V& v, int s) noexcept;
```

```
template<simd-type V> constexpr V bit_repeat(const V& v, int l) noexcept;
```

13 *Constraints:* The type `V::value_type` is an unsigned integer type ([[basic.fundamental](#)]).

¶ *Preconditions:* For `bit_repeat`, `l > 0` is true.

14 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v[i], s)` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-type V>
```

```
constexpr rebind_t<make_signed_t<typename V::value_type>, V> bit_width((
```

```
template<simd-type V>
```

```
constexpr rebind_t<make_signed_t<typename V::value_type>, V> countl_zer
```

```
template<simd-type V>
```

```
constexpr rebind_t<make_signed_t<typename V::value_type>, V> countl_one
```

```
template<simd-type V>
```

```
constexpr rebind_t<make_signed_t<typename V::value_type>, V> countr_zer
```

```
template<simd-type V>
constexpr rebind_t<make_signed_t<typename V::value_type>, V> countr_one(
template<simd-type V>
constexpr rebind_t<make_signed_t<typename V::value_type>, V> popcount(cc
```

15 *Constraints:* The type `V::value_type` is an unsigned integer type ([[basic.fundamental](#)]).

16 *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v[i])` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

```
template<simd-type V>
constexpr V bit_compress(const V& v, const V& m) noexcept;
template<simd-type V>
constexpr V bit_expand(const V& v, const V& m) noexcept;
```

¶ *Constraints:* The type `V::value_type` is an unsigned integer type ([[basic.fundamental](#)]).

¶ *Returns:* A `basic_vec` object where the i^{th} element is initialized to the result of `bit-func(v[i], m[i])` for all i in the range `[0, V::size())`, where `bit-func` is the corresponding scalar function from `<bit>`.

§ 5. References

[N5008] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-03-15)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5008.pdf>

[P3104R3] Jan Schultke. Bit permutations (2025-02-11)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3104r3.html>

[P2933R4] Daniel Towner, Ruslan Arutyunyan. Extend `<bit>` header function with overloads for `std::simd` (2025-02-13)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2933r4.html>